COURSE OUTLINE

Web Development

Frontend Development, Backend Development and Full-Stack Integration

Programme Duration	6 months (24 weeks)
Total Delivery	72 sessions (3 per week)
Prepared by	Femi Sodiq Oladele, Instructor

1. Course Overview

Course title	Web Development: Frontend, Backend and Full-Stack Integration
Duration	6 months, delivered in two 3-month phases
Frequency	3 sessions per week
Delivery	Frontend Track: 36 sessions Backend Track: 36 sessions Total: 72 sessions
Learning approach	Practical, project-based, progressive learning
Entry level	Beginner. No prior web development experience is assumed.
Core technologies	HTML5, CSS3, JavaScript, Git/GitHub, REST APIs, React, Vite, Node.js, Express.js, PostgreSQL, JWT, bcrypt

2. Course Description

This course takes students with little or no prior web development experience through the fundamentals of modern web development. It begins with how the Internet and the Web work, builds the skills needed to create interactive user interfaces in the first phase, and then teaches students to build server-side applications, databases and secure APIs in the second phase.

Frontend and backend concepts are connected progressively, so that students understand how a complete web application works rather than learning each technology in isolation. Every module follows a practical cycle and the course ends with a deployed, documented and defended full-stack project.

3. Course Aim and Learning Outcomes

Aim: to equip beginners to build, deploy and explain functional full-stack web applications.

On successful completion of the course, students will be able to:

1. Explain how the Internet, the Web, browsers, servers and web applications work.
2. Build semantic and accessible HTML structures.
3. Create responsive, well-structured interfaces with CSS.
4. Use JavaScript to create interactive web experiences.
5. Manipulate the DOM and handle browser events and forms.
6. Consume and work with REST APIs.
7. Build modern frontend applications with React.
8. Use Git and GitHub for version control and collaboration.
9. Explain HTTP, client-server communication and REST architecture.
10. Build backend applications and REST APIs with Node.js and Express.js.
11. Design and work with relational databases using PostgreSQL.
12. Implement CRUD operations, authentication and authorization.
13. Apply fundamental web security practices.

14. Deploy functional frontend and backend applications.
15. Complete, document, present and defend a practical full-stack project.

4. Target Students and Entry Requirements

- Target students: beginners with little or no web development experience.
- Prerequisites: none. Basic computer literacy (file management, using a browser and typing) is expected.
- Students should have regular access to a computer for practice and project work.

5. Programme Structure

Phase	Duration	Primary focus	Core stack
Phase 1	Months 1–3 (Weeks 1–12)	Frontend Development	HTML5, CSS3, JavaScript, Git/GitHub, REST APIs, React, Vite
Phase 2	Months 4–6 (Weeks 13–24)	Backend Development	JavaScript, Node.js, Express.js, PostgreSQL, REST APIs, Git/GitHub, JWT, bcrypt
Integration	Throughout / final project	Full-stack integration	Connecting frontend, API, database, authentication and deployment

6. Detailed Course Schedule

6.1 Phase 1: Frontend Development (Weeks 1–12, 36 sessions)

Week	Mod	Module	Key topics	Practical output
1	1	Web Development Fundamentals	Internet and Web concepts; browsers; client-server model; developer tools; tool setup	Working development environment; DevTools observation task
1–2	2	HTML	Syntax; semantic HTML; forms; tables; multimedia; accessibility	Personal Profile Website
3–4	3	CSS	Selectors; cascade; box model; typography; colours; Flexbox; Grid	Styled Profile Website
5	4	Responsive Web Design	Mobile-first design; media queries; responsive layouts	Responsive Landing Page
5	5	Git & GitHub	Version control; repositories; commits; branches; collaboration	All projects published on GitHub; pull request exercise
6	6	JavaScript Fundamentals	Variables; data types; operators; conditionals; functions	Practical coding exercises
7	7	JavaScript Core	Arrays; objects; loops; array methods; modern JavaScript	Data-handling exercises; start of interactive app

Week	Mod	Module	Key topics	Practical output
8	8	DOM & Browser Interaction	DOM manipulation; events; forms; validation	JavaScript Interactive Application
9	9	APIs & Asynchronous JavaScript	JSON; HTTP basics; Fetch; promises; async/await	API-Driven Web Application
10	10	React Fundamentals	JSX; components; props; state; events	Component-based exercises
11	11	React Application Development	Forms; effects; API integration; application structure	React Application
12	12	Frontend Capstone	Planning; development; deployment; documentation; presentation	Frontend Capstone Project (deployed and presented)

Where two modules share a week, they are delivered across that week's three sessions.

6.2 Phase 2: Backend Development (Weeks 13–24, 36 sessions)

Week	Mod	Module	Key topics	Practical output
13	1	Backend Fundamentals	Client-server architecture; servers; APIs; databases; request/response	Architecture diagram of a full-stack application
13–14	2	Node.js Fundamentals	Runtime; npm; modules; project structure	Node.js CLI Application
15	3	HTTP & Web Communication	Methods; headers; body; status codes; requests and responses	Basic HTTP Server
16	4	Express.js	Servers; routing; middleware; request/response handling	Express server with routes and middleware
17	5	REST API Development	REST principles; CRUD; endpoints; API design	Express REST API; CRUD API
18	6	Database Fundamentals	Database concepts; relational databases; SQL; data modelling	Entity-relationship design; SQL scripts
19	7	PostgreSQL Integration	Database connections; queries; relationships; persistence	PostgreSQL Database Application
20	8	API Validation & Error Handling	Input validation; centralised errors; API consistency	Validated API with consistent error responses
21	9	Authentication & Authorization	Registration; login; password hashing; JWT; protected routes	Authentication API
22	10	Backend Security	CORS; SQL injection; environment variables; secure practices	Security review of own API
23	11	Production & Deployment	Git; environment configuration; deployment; documentation	Deployed API with README
24	12	Backend Capstone	System design; implementation; deployment; project defence	Backend Capstone Project (deployed and defended)

7. Full-Stack Integration

Integration is built into the course rather than left to the end. Students first learn to consume ready-made APIs in the frontend phase (Modules 9 and 11). In the backend phase they build the API itself, connect it to a PostgreSQL database, secure it with authentication, and finally connect their frontend application to their own deployed backend. This gives students an end-to-end understanding of how requests move from the browser to the server and database and back.

8. Teaching and Learning Approach

Each topic follows a progressive cycle that moves students from concepts to independent implementation:

Concept → Practice → Mini Project → Integration → Capstone → Deployment → Project Defence

- Instructor-led demonstrations with live coding and visual explanations.
- Guided practical exercises in every session.
- Mini projects at the end of major modules.
- Version control with Git and GitHub from Week 5 onward.
- Code review, debugging practice and documentation as regular habits.
- Capstone presentation and defence at the end of each phase.

Session format

Session	Focus
Session 1	Concept introduction, demonstration and discussion
Session 2	Guided practice and exercises
Session 3	Project work, review and consolidation

9. Practical Projects

Frontend projects	Backend projects
Personal Profile Website	Node.js CLI Application
Responsive Landing Page	Basic HTTP Server
JavaScript Interactive Application	Express REST API
API-Driven Web Application	CRUD API
React Application	PostgreSQL Database Application
Frontend Capstone Project	Authentication API
	Backend Capstone Project

Frontend capstone themes: Student Management Dashboard, Expense Tracker, Task Management App, Course Management Dashboard, Weather App or Movie Explorer.

Backend capstone themes: Student Management API, E-commerce API, Task Management API, Course Management API, Inventory Management API or Event Management API.

10. Assessment

Assessment component	Weight	Description
Practical exercises	20%	In-session coding tasks
Assignments	15%	Take-home tasks after each module
Quizzes / concept checks	10%	Short checks on key concepts
Mini projects	15%	Module-level projects
Final capstone project	30%	Built, deployed and documented application
Project presentation	10%	Demonstration and defence of the project
Total	100%	

11. Skills Developed

Area	Skills
Web fundamentals	Internet and Web concepts, client-server architecture, HTTP, APIs
Frontend	HTML, CSS, responsive design, JavaScript, DOM, React
Backend	Node.js, Express.js, REST APIs, PostgreSQL, authentication, authorization
Tools	VS Code, Git, GitHub, browser DevTools, API testing tools
Professional skills	Problem solving, debugging, code organisation, version control, documentation, presentation, deployment

12. Resources and Requirements

Software

- VS Code, a modern browser (Chrome or Edge), Git and a GitHub account, Node.js (LTS), PostgreSQL, and an API testing tool such as Postman.

Reference materials

- MDN Web Docs (HTML, CSS, JavaScript).
- React documentation (react.dev).

- Node.js, Express.js and PostgreSQL official documentation.
- Instructor slide decks, code examples and practice exercises.